

Um guia moderno e simples para aprender a usar o GitHub na prática. Com o intuito de ajudar todos aqueles que estão começando, inclusive o próprio criador.



O que você vai aprender

- O que é GitHub?
 - Como criar conta?
 - Como criar repositórios?
 - Como baixar projetos?
 - Como enviar arquivos?
 - Como trabalhar em equipe?
 - Como usar GitHub no VS Code?
 - Como usar GitHub Desktop?
 - Conceitos importantes
 - Fluxo real de programadores
-



O que é GitHub?

O GitHub é uma plataforma onde programadores:

- guardam projetos,
 - compartilham código,
 - trabalham em equipe,
 - criam portfólio,
 - contribuem em projetos.
-



Pense no GitHub como:

Um Google Drive para programadores.

Mas muito mais poderoso.



Criando Conta no GitHub

Acesse:

<https://github.com>

Clique em:

Sign Up



Entendendo a Interface do GitHub

Página inicial

Você verá:

- seus repositórios,
 - atividade recente,
 - projetos,
 - contribuições.
-



O que é um Repositório?

Um repositório é a pasta do projeto no GitHub.

Exemplo:

```
meu-site  
api-node  
projeto-escola
```



Criando seu Primeiro Repositório

1. Clique no botão:

New Repository

2. Escolha o nome

Exemplo:

meu-primeiro-projeto

3. Escolha:

Público

Qualquer pessoa pode ver.

Privado

Só você ou pessoas autorizadas.

4. Clique em:

Create Repository



Estrutura do Repositório

Dentro do repositório você verá:

- arquivos,
- pastas,
- commits,
- histórico,
- colaboradores.

README.md

O README é a apresentação do projeto.

Exemplo:

```
# Meu Projeto
```

```
Projeto feito para aprender GitHub.
```

Enviando Arquivos pelo Site

Você pode enviar arquivos sem terminal.

Clique em:

Add File

Depois:

Upload Files

O que é Commit?

Um commit é um salvamento do projeto.

Exemplo:

```
adicionei login  
corrigi erro  
criei homepage
```



Fazendo Commit pelo GitHub

Depois de enviar arquivos:

1. Escreva uma mensagem
2. Clique em:

Commit Changes



O que é Branch?

Branch é uma versão paralela do projeto.

Serve para:

- testar ideias,
 - criar funcionalidades,
 - evitar quebrar o projeto principal.
-



Branch Principal

Normalmente:

main



Criando Branch no GitHub

1. Clique no nome da branch
2. Digite o nome novo
3. Clique em:

Create Branch



Pull Request

Pull Request é um pedido para juntar alterações.

Muito usado em equipes.



Exemplo

Criei uma funcionalidade nova.
Posso juntar ao projeto principal?

Isso é um Pull Request.



Trabalhando em Equipe

GitHub permite:

- convidar pessoas,
 - revisar código,
 - comentar linhas,
 - aprovar mudanças.
-



Issues

Issues servem para:

- bugs,
 - tarefas,
 - ideias,
 - organização.
-



Star

Dar Star significa:

Gostei do projeto.



Fork

Fork cria uma cópia de um projeto na sua conta.

Muito usado em:

- open source,
 - contribuições.
-



Clonando Projeto

Clonar significa baixar o projeto.

No GitHub

Clique em:

Code

Depois copie a URL.



GitHub no VS Code

O VS Code possui integração incrível com GitHub.

Você consegue:

- fazer commits,
- enviar arquivos,
- baixar alterações,
- trocar branches,
- tudo visualmente.



GitHub Desktop

Aplicativo oficial do GitHub.

Download:

<https://desktop.github.com>



Vantagens do GitHub Desktop

Excelente para iniciantes.

Você faz:

- commit,
- push,
- pull,
- branch,

clikando em botões.



Contributions

No seu perfil existe um gráfico de contribuições.

Cada quadrado representa atividade.



GitHub como Portfólio

Hoje empresas analisam GitHub.

Seu perfil pode mostrar:

- projetos,
- organização,
- evolução,
- habilidades.



Boas Práticas



Use README

Explique:

- objetivo,
 - tecnologias,
 - instalação.
-



Faça commits organizados

Bom:

```
feat: sistema de login
```

Ruim:

```
arrumei coisas
```



Organize projetos

Use:

- pastas,
 - nomes claros,
 - documentação.
-



Fluxo REAL no GitHub

```
Criar projeto
```

```
↓
```

Enviar arquivos
↓
Commit
↓
Atualizar projeto
↓
Criar branch
↓
Pull Request



Conceitos MAIS IMPORTANTES

Conceito	Significado
Repository	projeto
Commit	salvamento
Branch	versão paralela
Pull Request	pedido de alteração
Clone	baixar projeto
Fork	cópia do projeto
Issue	tarefa/bug



O que Programadores Mais Fazem no GitHub?

- publicar projetos,
- trabalhar em equipe,
- estudar código,
- criar portfólio,
- contribuir em open source.



Comandos Git na Prática — Guia Profundo para Iniciantes

Aqui você vai aprender os comandos Git que programadores realmente usam no dia a dia, entendendo não apenas o comando, mas o motivo de usar cada um.



Primeiro: Como o Git Funciona Mentalmente

Antes dos comandos, entenda isto:

Seu projeto passa por diversas etapas.



As 3 áreas principais do Git

1. Working Directory

É onde você edita os arquivos.

Exemplo:

Você altera `index.html`

Mas o Git ainda não salvou isso.

2. Staging Area

Área temporária.

Você diz ao Git:

"Quero salvar essas alterações."

Isso acontece com:

`git add`

3. Repository

Aqui o commit realmente é criado.

Isso acontece com:

```
git commit
```

Fluxo Mental do Git

```
Editar arquivo  
↓  
git add  
↓  
git commit  
↓  
git push  
↓  
GitHub
```

git status — O comando MAIS importante

Se existe um comando obrigatório no Git, é este:

```
git status
```

O que ele faz?

Ele mostra:

- arquivos modificados,
 - arquivos novos,
 - arquivos adicionados,
 - branch atual,
 - se há commits pendentes,
 - se o projeto está atualizado.
-



Exemplo real

```
git status
```

Resultado:

```
On branch main
Changes not staged for commit:
  modified:   index.html
```



Tradução disso

Você modificou index.html,
mas ainda não usou git add.

+ git add — Preparando arquivos

O Git não salva automaticamente.

Você precisa escolher o que vai entrar no commit.



Adicionar tudo

```
git add .
```

O ponto (.) significa:

Tudo da pasta atual.



Adicionar arquivo específico

```
git add index.html
```



Exemplo real

Você alterou:

```
index.html  
style.css  
app.js
```

Mas quer salvar apenas um arquivo?

```
index.html
```

Então, faça:

```
git add index.html
```



O que significa “staged”?

Significa:

```
"Pronto para commit"
```



git commit — Criando salvamentos

Commit é um ponto salvo do projeto. E como se voce tirasse uma foto dos seus filhos, ao longo da vida deles.



Estrutura

```
git commit -m "mensagem"
```

Exemplo real

Preste bastante atencao na hora de nomear os seus commits, eles irao guiar o futuro do seu projeto.

```
git commit -m "feat: sistema de login"
```

Tradução mental

```
"Git, salve esta versão do projeto."
```

Commits precisam ser claros

Ruim

```
arrumei coisas
```

Bom

```
feat: adicionando autenticação JWT
```

Tipos comuns de commit

Tipo	Significado
feat	nova funcionalidade
fix	correção

Tipo	Significado
docs	documentação
refactor	reorganização
style	formatação
test	testes

git push — Enviando para GitHub

Depois do commit:

Ainda está só no seu computador.

Para enviar:

```
git push
```

Tradução mental

Seu computador → GitHub

Exemplo real

```
git push origin main
```

O que significa?

Parte	Significado
push	enviar
origin	GitHub

Parte	Significado
main	branch

git pull — Atualizando projeto

Baixa mudanças do GitHub.

```
git pull
```

Tradução mental

```
GitHub → Seu computador
```

Situação real

Seu colega enviou mudanças.

Você usa:

```
git pull
```

para atualizar seu projeto.

git clone — Baixando projetos

Clonar significa:

```
Baixar um projeto completo do GitHub.
```

Estrutura

```
git clone URL
```



Exemplo real

```
git clone https://github.com/user/projeto.git
```



Depois entre na pasta

```
cd projeto
```



git branch — Trabalhando com branches

Branches permitem criar versões paralelas.



Ver branches

```
git branch
```



Resultado

```
* main  
  login
```

O * significa:

```
Branch atual.
```

Criar branch

```
git branch login
```



Trocar branch

```
git switch login
```



Criar e trocar automaticamente

```
git switch -c login
```



Por que usar branches?

Porque você pode:

- testar funcionalidades,
 - evitar quebrar a main,
 - trabalhar em equipe.
-



git log — Histórico do projeto

Mostra commits.

```
git log
```



Exemplo

```
commit 9a8d7f6
Author: Gabriel Silva
feat: sistema de login
```

git remote — Conexão com GitHub

Mostra conexão do projeto com GitHub.

```
git remote -v
```

Exemplo

```
origin https://github.com/user/projeto.git
```

O que é origin?

Normalmente:

```
origin = GitHub
```

git diff — Ver mudanças

Mostra alterações antes do commit.

```
git diff
```

Muito útil para:

- revisar código,
- encontrar erros,

- entender mudanças.
-



git restore — Desfazer alterações

Descarta mudanças.

```
git restore index.html
```



Cuidado

Isso apaga alterações não commitadas.



Fluxo REAL de Programador

Todos os dias

```
git pull
```

Atualiza projeto.

```
git status
```

Verifica situação.

```
git add .
```

Adiciona alterações.

```
git commit -m "feat: dashboard financeira"
```

Cria commit.

```
git push
```

Envia para GitHub.

Fluxo Visual Completo

```
Editar código  
↓  
git status  
↓  
git add .  
↓  
git commit  
↓  
git push  
↓  
GitHub atualizado
```

Erros MAIS comuns

nothing to commit

Nada mudou.

not a git repository

Você não está em um projeto Git.

Authentication failed

Problema de login/token.

pathspec did not match

A branch não existe.



Dicas MUITO importantes



Sempre use git status

Ele praticamente explica o projeto.



Faça commits pequenos

Melhor:

```
Vários commits organizados.
```

Do que:

```
Um commit gigante.
```



Use branches

Evita quebrar projeto principal.



Commits claros

Commits contam a história do projeto.

Resumo Final

GitHub é:

- plataforma de projetos,
 - rede social de programadores,
 - portfólio,
 - ferramenta de colaboração.
-

Parabéns!

Agora você já entende o funcionamento essencial do GitHub de forma prática e intuitiva.

Com o tempo, tudo isso ficará natural no seu fluxo de desenvolvimento.